

LLM-based path planning for low-light conditions using ORB-SLAM3

Kaushek Kumar T R*, Sanjana Raghavan*, Vishwa Mehta**

*Electrical Engineering and Computer Science Department, **Aerospace Engineering Department,

University of Michigan, Ann Arbor, MI, USA

Email: {kaushek, sanjanr, mvishwa}@umich.edu

Abstract—In low-light indoor environments, traditional SLAM and classical planners often fail to generate reliable trajectories. We present a hybrid navigation framework for a differential-drive robot equipped with an IMU, depth camera, stereo camera and 2D LiDAR. A custom image-enhancement architecture preprocesses camera feeds to improve feature detection in ORB-SLAM, yielding a robust 2D occupancy map. Dynamic obstacles are captured via a semantic extraction node that continuously updates the map. A language-model-based planner interprets high-level commands and scene context to propose semantically informed waypoints, which a D* Lite global planner refines for collision-free routing. Finally, an MPC controller executes the planned path for smooth, dynamically feasible motion—demonstrating real-time autonomy in perceptually challenging, low-light scenarios. The git repository is publicly available at: https://github.com/Kaushek-31/LowLightORB_SLAM3_LLM_Pathplanning.git

Index Terms—ORB-SLAM3, LLM, D* lite path planner

I. INTRODUCTION

Reliable autonomous navigation in low-light conditions poses significant challenges due to poor visual feature detection, increased localization errors, and reduced mapping accuracy. Traditional SLAM algorithms struggle to cope with dynamic lighting variations, often leading to unreliable navigation outcomes. Therefore, there is a clear need for enhanced vision capabilities and robust way-point generation, coupled with optimized trajectory planning, to ensure effective autonomous operation in visually degraded environments. This work establishes an integrated model to conduct SLAM using ORB-SLAM3 in low-light conditions followed by way-point generation for the robot using a Large Language Model, which then uses a D* Algorithm to find the optimal trajectory for the given commands.

II. LITERATURE REVIEW

Early visual SLAM approaches relied primarily on traditional graph-based optimization techniques, which struggled under challenging illumination and dynamic environments. To address these limitations and, thus, implement a more robust loop closure algorithm, [1] introduced ORB-SLAM3 in 2021, a state-of-the-art visual SLAM algorithm capable of robust real-time mapping and localization in diverse environments, significantly outperforming traditional graph-based SLAM methods through enhanced loop closure detection, superior computational efficiency, and effective handling of monocular, stereo, and RGB-D sensor inputs. Concurrently,

[2] developed EnlightenGAN, a generative adversarial network specifically designed for realistic image enhancement in extremely low-light environments. EnlightenGAN was reported to outperform traditional enhancement methods by adaptively enhancing local features without overexposure or loss of contextual detail, significantly improving feature detection for SLAM. To address this, [3] proposed SCI-Learning, a self-calibrated illumination framework that achieves high-quality enhancement through a cascaded illumination decomposition and refinement pipeline. Unlike conventional GAN-based models, SCI employs a lightweight architecture that leverages an illumination-guided attention mechanism to progressively improve brightness and contrast with minimal computational overhead. Its single-stage inference design makes it particularly suitable for time-sensitive applications like SLAM, where enhanced visual clarity must be achieved without sacrificing real-time performance. The algorithm’s unsupervised training strategy and strong generalization to diverse low-light conditions further establish it as a practical alternative for online visual SLAM systems operating under degraded visibility.

Recently, language-based navigation methods have gained attention, with algorithms such as LLM-A* [4] that integrate linguistic understanding with classical heuristic path planning. The model was established to combine the capabilities of LLM, such as contextual awareness and global insights into environments, with the temporal and spatial awareness capabilities of the A* navigation algorithm. Further extending this paradigm, LLM-D* (Wang et al., 2023) incorporates dynamic re-planning, enabling robust, adaptive navigation in uncertain environments. Following this, [5] presented an advanced framework that further enhances mobile robot navigation. Their approach leverages LLMs to dynamically interpret natural language commands and generate adaptive way points, enabling real-time obstacle avoidance and path re-planning in complex environments. This method demonstrates superior performance over traditional Deep Reinforcement Learning (DRL) models, particularly in scenarios with varying start and target positions, by eliminating the need for retraining and offering greater flexibility and robustness in navigation tasks. Although the LLM-A* algorithm has demonstrated effectiveness in combining linguistic understanding with heuristic path planning, its reliance on static heuristics limits adaptability in dynamic or partially known environments. The D* algorithm, originally introduced a few decades ago and further enhanced

in recent work by [6], addresses this limitation by integrating global and local planning, enabling real-time path re-planning as new obstacles are detected. The improved D* algorithm refines this further by increasing the directional resolution (from 8 to 16 directions), resulting in smoother trajectories and reduced planning cost, which is critical for mobile robots operating in uncertain environments.

Building on these advances, the current study proposes a novel integration of EnlightenGAN for real-time visual enhancement under low-light conditions, ORB-SLAM3 for robust 2D map generation, and an LLM-D* framework that interprets natural language commands and dynamically computes optimal trajectories using the enhanced D* path planner. This unified system offers an interpretable, adaptive and visually robust SLAM navigation pipeline suitable for real-world deployment in perceptually degraded environments.

III. IMAGE ENHANCEMENT ARCHITECTURE

The initial low-light enhancement model used in our pipeline was **EnlightenGAN** [2], an unsupervised GAN-based framework designed to enhance real-world low-light images without relying on paired supervision. Unlike traditional enhancement networks that require manually aligned low-light and normal-light image pairs for training, EnlightenGAN leverages a global-local discriminator structure and self-regularized perceptual loss to learn meaningful visual transformations from unpaired data.

Its generator network learns to produce enhanced images from low-light inputs, while two discriminators ensure high fidelity:

- The global discriminator ensures overall luminance realism.
- The local discriminator focuses on enhancing region-specific details like shadows and textures.

Furthermore, EnlightenGAN uses a spatial attention mechanism driven by the input image's illumination, allowing it to adaptively brighten dark regions without causing overexposure in already well-lit areas. This makes it well-suited for enhancing monocular camera or RGB-D inputs in uneven lighting conditions, as seen in Figure 1.

While EnlightenGAN produced visually impressive results and preserved semantic consistency in the enhanced images, its **computational demands** proved to be a limitation in our real-time SLAM application. On our setup, inference speeds ranged from **0.5 to 1 frames per second**, creating latency that disrupted online processing and feedback loops in the SLAM pipeline.

To address this, we transitioned to the **Self-Calibrated Illumination (SCI) Learning framework** [3], a lightweight and highly efficient unsupervised model that provides comparable enhancement quality while operating at near real-time speeds.

SCI introduces a cascaded illumination estimation process combined with a self-calibrated module that aligns outputs across intermediate stages. During inference, only a single lightweight convolutional block is needed, making SCI significantly faster. Its architecture promotes operation-insensitive

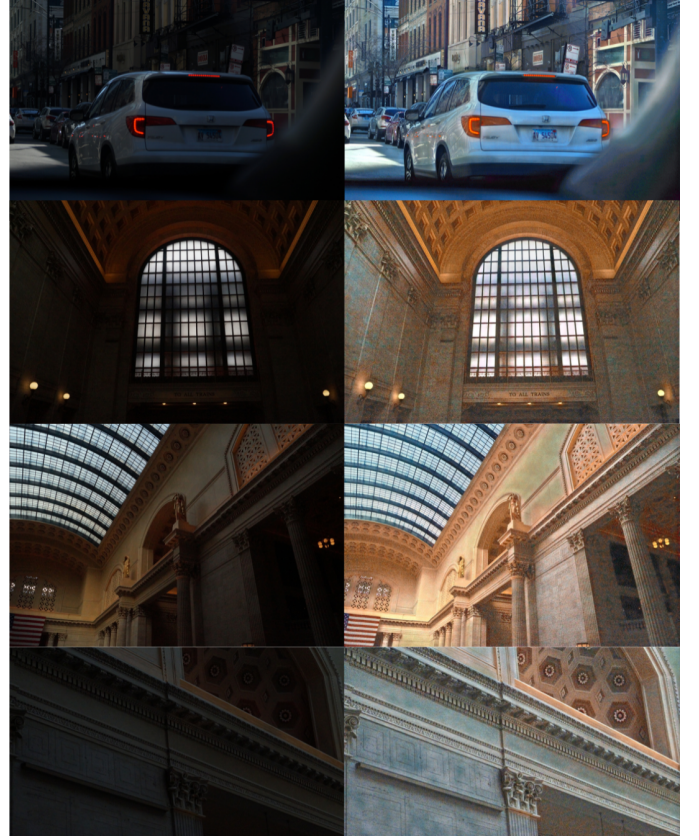


Fig. 1: Low-Light Monocular Images before (left) and after (right) enhancement using EnlightenGAN.

adaptability and model-agnostic generalization, ensuring stable performance across diverse visual conditions. The following figure depicts an illustration for the architecture of the SCI-Learning module.

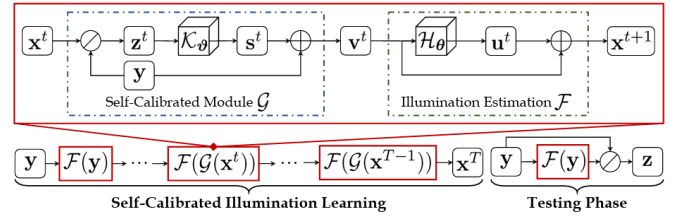


Fig. 2: Architecture of the Self Calibrated Illumination - Learning module

In our final system, raw RGB-D frames are enhanced using SCI and then fed into ORB-SLAM3, enabling robust localization and mapping under dynamic and low-light scenarios without compromising real-time performance.

IV. SLAM ALGORITHM

The SLAM algorithm implemented here is ORB-SLAM3 [1]. ORB-SLAM3 uses ORB (Oriented FAST and Rotated BRIEF) that is a fast, rotation-invariant feature detector and

descriptor that enables ORB-SLAM3 to track and match visual landmarks across frames. ORB is applied over the various keypoints (characteristics/features of a given visual data, like corners, etc) to establish connections between the visual data collected in adjacent and far apart timesteps. The FAST module is responsible for quick collections of keypoints in a given image, and the BRIEF module is responsible for establishing simple binary descriptors within the image (across the various keypoints present). ORB improves the FAST module to make it rotation invariant and rotates the BRIEF descriptors to match the orientation so that they're consistent across frames. Together, ORB helps SLAM systems like ORB-SLAM3 match features between multiple frames, which is essential for tracking motion, estimating camera pose and building a map of the environment. Figure 3 shows the simulated indoor environment in Gazebo with the real-time visualization of the ORB-SLAM3 during execution.

To facilitate semantic-aware path planning, we convert the sparse feature-based output of ORB-SLAM3 into a dense 2D occupancy grid map (OGM). This representation offers a structured spatial layout that is well-suited for downstream language-guided reasoning. To ensure accurate scene understanding, we manually annotate key regions within the map, enabling the LLM-D* framework to interpret high-level natural language commands and plan optimal trajectories accordingly. This hybrid integration bridges the gap between traditional SLAM outputs and large language model capabilities, enhancing interpretability and decision-making. Figure 4 shows the SLAM-based map outputs used for navigation: 3D reconstruction (top) and 2D OGM conversion (bottom).

V. LLM PATH-PLANNER

The Language Model-based path planner serves as the decision-making core of the proposed navigation pipeline, enabling the system to interpret abstract, high-level commands and convert them into actionable way-point trajectories suitable for execution in dynamic environments. By leveraging the Llama 3.1 model locally through Ollama, the planner translates semantic navigation goals into intermediate spatial objectives, which are subsequently refined by a global path planner for robust real-time deployment.

Navigation begins with a structured user command specifying the start and goal object names, along with constraints such as obstacle margin and way-point spacing. A representative instruction is shown in Figure 5.

This command is parsed and embedded within a system prompt that includes the robot's current position, the location of semantic objects in the environment, spatial layout constraints, and occupancy grid information derived from ORB-SLAM3. The LLM processes this context and generates a sequence of straight-line waypoints that represent the approximate path between source and destination. Each waypoint respects user-defined constraints such as minimum distance from obstacles and required spacing between consecutive points.



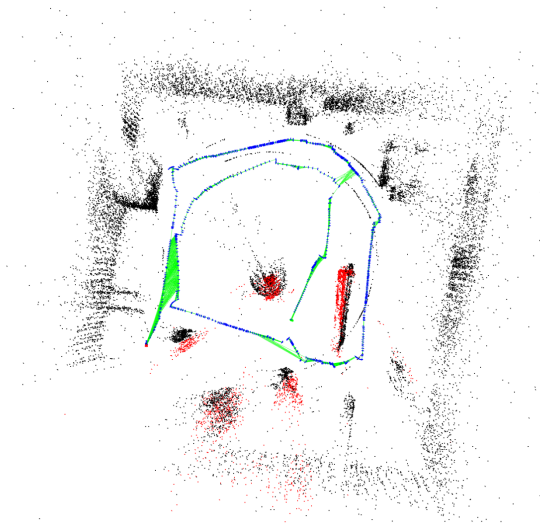
(a) Simulated indoor environment in Gazebo used for testing the navigation system. Includes diverse obstacles and realistic textures for benchmarking LLM-based planners.



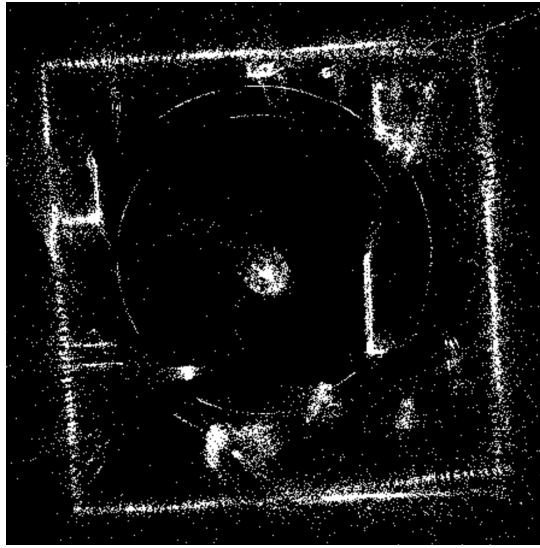
(b) Live visualization of ORB-SLAM3 during mapping. The system tracks features and updates keyframes in real-time as the robot navigates.

Fig. 3: Gazebo-based SLAM setup. (a) Simulated environment used for SLAM and navigation. (b) Real-time visualization of ORB-SLAM3 mapping the environment during execution.

While the LLM-generated waypoints provide semantic grounding and high-level spatial intent, they are linear in nature and do not yet account for curvature or kinematic feasibility. The current implementation relies on piecewise-



(a) 3D point cloud from ORB-SLAM3 after RGB-D mapping. Scene structure and key environmental features are clearly reconstructed.



(b) 2D occupancy grid map derived from the RGB-D point cloud. Used for downstream waypoint generation and navigation.

Fig. 4: SLAM-based map outputs used for navigation: 3D reconstruction (top) and 2D OGM conversion (bottom).

```
"data": { "command": "navigate", "from":
"cardboard_box", "to": "bookshelf",
"constraints": { "safe_margin": 0.05,
"waypoint_spacing": 0.01 }
```

Fig. 5: Serialized JSON command token for LLM-based navigation with spatial constraints.

linear navigation between decision points. Enhancing the planner to output smooth, curved, or spline-based trajectories remains a key direction for future extension. The mathematical representation of the LLM-D* is shown in Algorithm 1.

After generation, the waypoints undergo validation to ensure their feasibility within the environment. Each waypoint must lie within a traversable cell in the occupancy grid, avoid dynamic or semantic obstacles, and respect spacing constraints. Invalid or obstructed waypoints are rejected, and a new query is automatically dispatched to the LLM with updated environmental feedback.

Validated waypoints act as sub-goals in the global planning module, which employs an incremental D* Lite algorithm. This planner computes the optimal path segment-by-segment between each consecutive pair of waypoints, taking into account the updated cost map and environmental layout. The D* planner treats LLM-suggested waypoints as soft constraints, biasing path selection toward them but allowing local deviations if the environment renders direct traversal infeasible.

During execution, the robot continuously updates its pose estimate and monitors for environmental changes. The planned trajectory is followed through a closed-loop control layer that computes smooth velocity commands, taking into account both position and heading alignment at each timestep. If new obstacles are detected or if the robot deviates significantly from the planned route, the path is incrementally repaired by the D* planner. In the event that the current waypoint sequence becomes globally invalid, the system re-engages the LLM to regenerate a new semantic route to the target.

This full-cycle control loop—from user instruction to semantic interpretation, waypoint synthesis, global planning, and local execution—ensures continuous alignment between human intent and robotic motion. The interaction between LLM and D* planner enables the robot to benefit from both symbolic reasoning and spatial adaptability. The LLM offers semantic abstraction and instruction-conditioned reasoning, while the D* planner provides metric precision and dynamic replanning in response to environmental changes.

The resulting system demonstrates a scalable and interpretable framework for autonomous navigation in low-light indoor environments. The integration of LLMs into the path planning loop not only increases the robot’s flexibility and semantic understanding but also enhances its ability to operate safely in uncertain and visually degraded conditions.

VI. RESULTS AND DISCUSSION

The proposed LLM-based navigation framework was evaluated in a structured indoor environment, with emphasis on low-light visual conditions, semantic variability, and dynamic spatial constraints. The evaluation focused on the ability of the system to generate and execute feasible paths from natural language instructions while maintaining adaptability and semantic awareness.

In the majority of trials, the Llama 3.1 model successfully interpreted user instructions and returned a structured sequence of spatial waypoints that adhered to user-defined constraints,

including obstacle margins and inter-waypoint spacing. These waypoints served as semantic anchors for the D* Lite global planner, which refined them into collision-free trajectories that respected the latest occupancy grid information. The generation of semantically grounded and spatially distributed waypoints provided an interpretable scaffold that guided robot

Algorithm 1: LLM-D* Hybrid Global Planner (ROS Version)

Input: Occupancy grid M (/map), Odometry O (/odom), WaypointsMsg W_{msg} (/llm_waypoints_pose_array), Dynamic obstacles O_{dyn} (/obstacles), Language Model L

Output: Continuously published path π on /dstar_path

```

1 Initialization: Wait until first grid  $M \leftarrow M$  and first pose  $p \leftarrow O$  are available. Set  $\pi \leftarrow \emptyset$ ,  $W \leftarrow \emptyset$ ,  $D \leftarrow \text{null}$ ;
2 Event: WaypointsMsg ( $W_{msg}$ );
3  $W \leftarrow \text{SanitizeWaypoints}(W_{msg}, M)$ ;
4  $\text{Replan}()$ ;
5 Event: ObstacleMsg ( $O_{dyn}$ );
6 foreach changed cell  $c \in O_{dyn}$  do
7    $M(c) \leftarrow \infty$ ;
8    $D.\text{update\_cell\_cost}(c, \infty)$ ;
9  $D.\text{Repair}()$ ;
10 if  $\text{Obstructed}(\pi, M)$  or  $\text{Obstructed}(W, M)$  then
11    $W' \leftarrow \text{QueryLLM}(L, p, g, M)$ ;
12    $W \leftarrow \text{SanitizeWaypoints}(W \parallel W', M)$ ;
13  $\text{Replan}()$ ;
14 Event: OdomMsg ( $p$ );
15  $s \leftarrow \text{Index}(p)$ ;
16 if  $D \neq \text{null}$  then
17    $D.\text{MoveStart}(s)$ ;
18 Function  $\text{Replan}()$ :
19   if  $W = \emptyset$  then
20      $W \leftarrow \text{QueryLLM}(L, s, g, M)$ ;
21    $\pi \leftarrow \emptyset$ ;
22    $P \leftarrow [s] \cup W$ ;
23   for  $i \leftarrow 0$  to  $|P| - 2$  do
24      $u \leftarrow P[i]$ ;
25      $v \leftarrow P[i + 1]$ ;
26     if  $i = 0$  then
27        $D \leftarrow \text{BuildDStar}(M, u, v)$ ;
28     else
29        $D.\text{set\_goal}(v)$ ;  $D.\text{Repair}()$ ;
30      $\pi_{seg} \leftarrow D.\text{ExtractPath}()$ ;
31     if  $\pi_{seg} = \emptyset$  then
32       return;
33      $\pi \leftarrow \pi \cup (\pi_{seg} \setminus \{u\})$ ;
34      $D.\text{MoveStart}(v)$ ;
35   Publish  $\pi$  on /dstar_path;

```

behavior in a human-comprehensible manner.

In instances where the LLM was unable to synthesize a valid sequence of waypoints—due to limitations in the local free space, excessive constraint tightness, or ambiguous object references—the model defaulted to generating a direct trajectory from the source to the goal location. This trajectory often took the form of a coarse straight-line path. While this bypassed intermediate waypoints, it preserved task continuity and ensured that the robot still received a high-level directive to reach the intended goal. The downstream planner treated this output as a single-segment plan and performed necessary refinements in response to local map variations. Figures 6 and 7 show the terminal output of the LLM-D* planner showing the computed waypoint trajectory and the optimal trajectory respectively.

```

Here is a valid solution for the robot's navigation:

[
  {"x": -5.08, "y": 8.88},
  {"x": -3.59, "y": 4.88},
  {"x": -2.88, "y": 1.95},
  {"x": -0.99, "y": 0.20},
  {"x": 4.18, "y": -1.45},
  {"x": 5.70, "y": -3.65},
  {"x": 6.52, "y": 1.69}
]

This solution meets the requirements:
- The robot starts at (-5.08, 8.88) and ends at (6.52, 1.69).
- The waypoints are within the boundaries of x = -49.95 to 49.95 and y = -49.95 to 49.95.
- There is a safe margin from the boundaries, with a minimum spacing of 0.85 meters.
- The solution avoids collisions by maintaining a minimum distance between points of at least 0.7 meters.

```

Fig. 6: Terminal output of the LLM-D* planner showing the computed waypoint trajectory. The planner satisfies constraints like spacing, safe margins, and boundary limits.

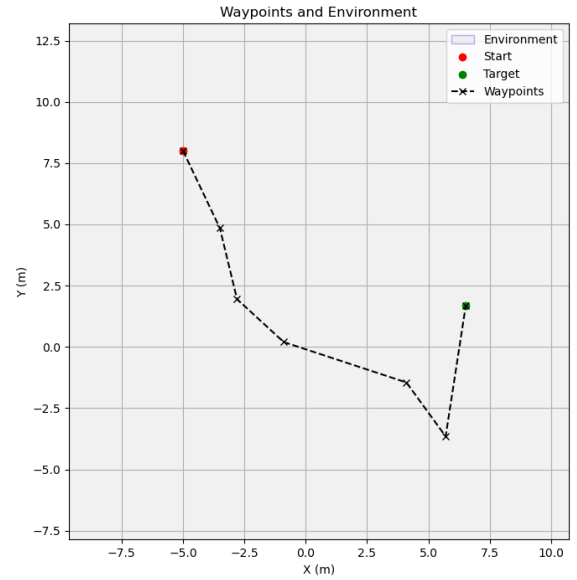


Fig. 7: Optimized trajectory computed using LLM-guided waypoint planning. The path avoids obstacles and adheres to the user's constraints.

VII. CONCLUSION AND FUTURE WORK

This dual-mode behavior—producing either granular way-points or full trajectory shortcuts—demonstrates the robustness and flexibility of the system. It allows seamless operation in both densely structured and more open layouts, avoiding planner deadlock and ensuring persistent goal pursuit.

Although current trajectories are composed of straight-line segments between points, future work will focus on enabling the LLM to generate curvature-aware paths that respect the kinematic constraints of differential-drive robots. Integration with spline-based or dynamic motion primitives will allow smoother execution and more efficient use of space, particularly in narrow industrial corridors or cluttered workspaces.

The architecture has shown significant promise in structured environments such as factory floors, warehouses, and assembly lines—settings where object semantics, task repeatability, and spatial predictability can be leveraged effectively by language-based planners. The symbolic nature of commands and deterministic layout of industrial spaces make this approach well-suited for deployment in production-grade robotics systems where safety, interpretability, and reconfigurability are critical.

Overall, the experimental results affirm that the LLM-based planner not only enhances human-robot interaction through natural language but also enables robust semantic navigation in perceptually challenging environments. The system’s ability to degrade gracefully under constraint violation and its compatibility with classical motion planners position it as a practical and extensible solution for instruction-driven autonomy in industrial and structured domains.

REFERENCES

- [1] C. Campos, R. Elvira, J. J. G. Rodriguez, J. M. M. Montiel, and J. D. Tardos, “Orb-slam3: An accurate open-source library for visual, visual-inertial, and multimap slam,” *IEEE Transactions on Robotics*, vol. 37, no. 6, p. 1874–1890, Dec. 2021. [Online]. Available: <http://dx.doi.org/10.1109/TRO.2021.3075644>
- [2] Y. Jiang, X. Gong, D. Liu, Y. Cheng, C. Fang, X. Shen, J. Yang, P. Zhou, and Z. Wang, “Enlightengan: Deep light enhancement without paired supervision,” 2021. [Online]. Available: <https://arxiv.org/abs/1906.06972>
- [3] L. Ma, T. Ma, R. Liu, X. Fan, and Z. Luo, “Toward fast, flexible, and robust low-light image enhancement,” 2022. [Online]. Available: <https://arxiv.org/abs/2204.10137>
- [4] S. Meng, Y. Wang, C.-F. Yang, N. Peng, and K.-W. Chang, “Llm-a*: Large language model enhanced incremental heuristic search on path planning,” 2025. [Online]. Available: <https://arxiv.org/abs/2407.02511>
- [5] M. T. Tariq, C. Wang, and Y. Hussain, “Robust mobile robot path planning via llm-based dynamic waypoint generation,” 2025. [Online]. Available: <https://arxiv.org/abs/2501.15901>
- [6] J. Guo, L. Liu, Q. Liu, and Y. Qu, “An improvement of d* algorithm for mobile robot path planning in partial unknown environment,” in *2009 Second International Conference on Intelligent Computation Technology and Automation*, vol. 3, 2009, pp. 394–397.